Modification d'un entrepôt de données

Gattano César

9 septembre 2026

Avant-propos

Ce rapport a été rédigé en usant de l'intelligence artificielle. Par soucis de transparence, les parties purement rédigées par l'IA sont signalées via une couleur de police bleu sombre. L'auteur certifie que l'ensemble du contenu générée par l'IA a été revu et validé par ses soins.

Voici un exemple : Je suis un assistant IA conçu pour vous aider dans la rédaction de votre rapport. Je génère du texte structuré et cohérent, que vous pouvez ensuite affiner. Mon rôle est de vous fournir un cadre de travail, tout en vous laissant le contrôle final. Ensemble, nous pouvons créer un document clair et professionnel.

En outre, l'auteur indique que l'IA générative a également été utilisée en auto-complétion durant la rédaction de ce rapport. Cette fonctionnalité ayant été utilisée tout au long du rapport, aucun indicateur n'a été mis en place pour souligner cet aspect, si ce n'est cette déclaration.

Table des matières

1 Contexte **2**

2 Préparation de la maintenance **2**

2.1 État initial de l'entrepôt de données 2

2.2 Planification de la maintenance 3

2.3 Suivi du service 3

1 Contexte

Je suis data engineer junior au sein de ShopFlow, une ESN spécialisée dans les solutions data pour le secteur e-commerce. Mon client, la société VenteRapide, exploite une boutique en ligne généraliste présente sur le marché français depuis 2021.

VenteRapide dispose d'un entrepôt de données (data warehouse) opérationnel, construit sur `PostgreSQL`et alimenté via `DBT`. Cet entrepôt centralise les données de commandes, de produits, de clients et de visites du site, et est utilisé quotidiennement par l'équipe analytique pour ses reportings.

Dans le cadre d'une évolution de l'activité, VenteRapide me confie la mission de maintenir et faire évoluer cet entrepôt en conditions opérationnelles.

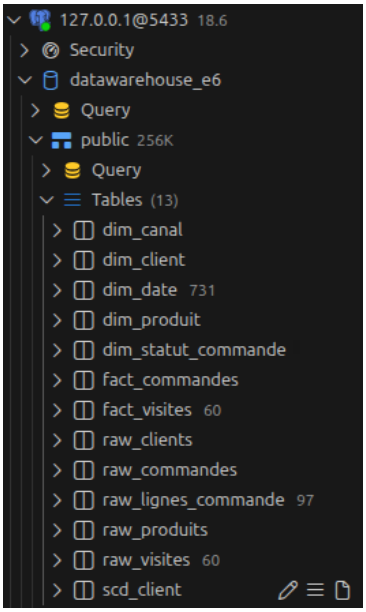


FIGURE 2 – Entrepôt initial

2 Préparation de la maintenance

2.1 État initial de l'entrepôt de données

L'entrepôt initial contient :

— 5 tables de données brutes : `clients`, `commandes`, `lignes_commandes`, `produits`, `visites`,

— 5 tables de dimensions : `canal`, `client`, `date`, `produit`, `statut_commandes`,

— 2 tables de faits : `commandes`, `visites`.

— 1 table SCD type 2 : `client`.

Le linéage des données est assuré par `DBT`, qui orchestre les transformations et les chargements. La figure 1 montre le linéage des données dans l'entrepôt initial.

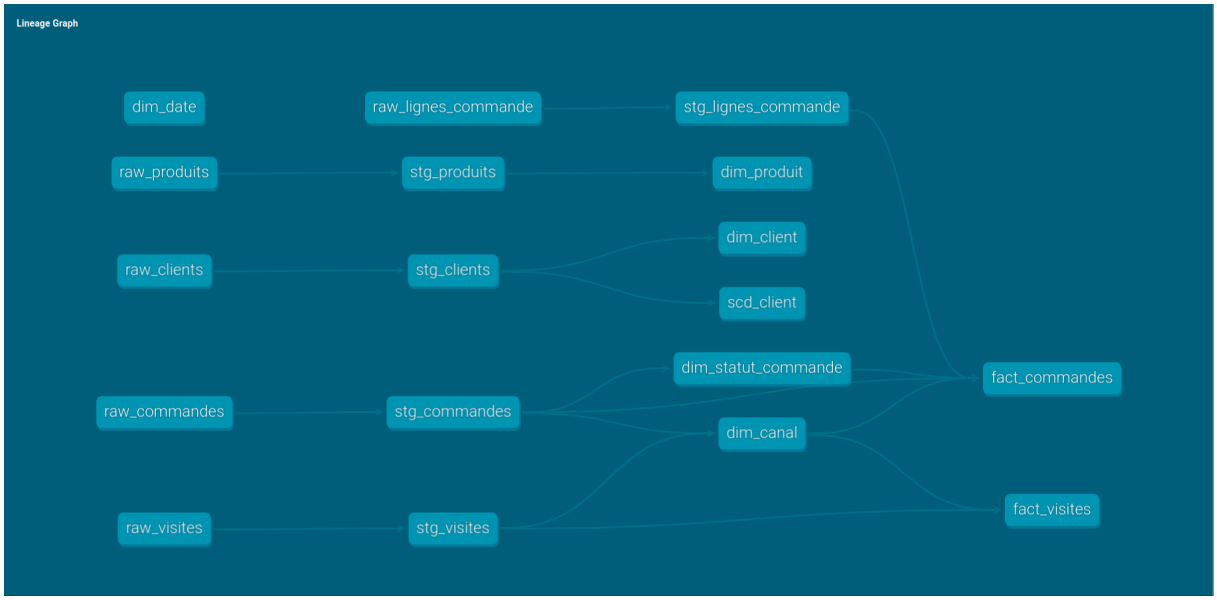
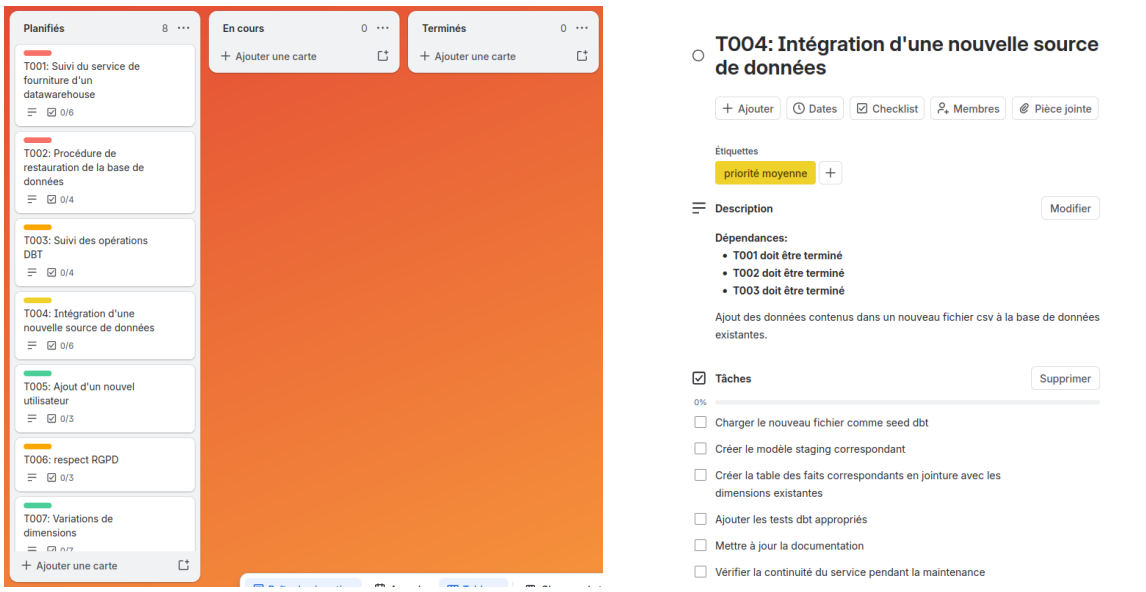


FIGURE 1 – Linéage des données dans l'entrepôt initial

2.2 Planification de la maintenance

La maintenance de l'entrepôt de données est planifiées grâce à un système de tickets. La vue d'ensemble et un exemple de tickets sont présentés en figure 3



(a) Vue du tableau (b) Exemple de ticket de maintenance

FIGURE 3 – Système de tickets de maintenance.

2.3 Suivi du service

Un SLA (Service Level Agreement) est un engagement formel entre un fournisseur de service et ses utilisateurs, définissant le niveau de qualité de service attendu, mesurable par des indicateurs précis, les SLI (Service Level Indicators). Il répond à trois questions : quoi (quel service), combien (quel niveau de performance), et que se passe-t-il en cas de non-respect.

Dans le cas d'une base de données, un SLA pourrait inclure des engagements sur :

— La disponibilité du service sur une période donnée : par exemple, un engagement de 99,9% de disponibilité mensuel.
— Le temps de traitement des incidents : délai de prise en charge et délai de résolution pouvant être fonction du niveau de criticité
— La planification de fenêtre de maintenance : plages de temps où la contrainte de disponibilité peut être levée.
— Recovery point objective (RPO) et Recovery time objective (RTO) : les indicateurs précisant la quantité maximale de perte de données et le temps maximal de restauration acceptable, en cas de rupture de service.
— Performance : temps de réponse des requêtes, nombre de transactions par seconde, etc.

Dans le cadre de ce projet, je fais le choix de mettre en place les SLI suivants :

— Mesure (booléenne) de la santé du conteneur `PostgreSQL` : simple tentative de connexion régulière pour vérifier que la base de données est accessible.
— Mesure de la latence d'une requête simple sur les tables de faits de la base de données (tables de haute valeur pour les équipes métiers).

Le script `scripts/set_monitoring_db.sh` permet de créer la base des données de monitoring et l'utilisateur associé, ainsi que les droits nécessaires via des requêtes SQL envoyés au sein du conteneur `PostgreSQL`. Le script `monitor.py` contient le code de sur-

veillance de la base de données qui écrit à intervalle régulier les mesures décrites ci-dessus dans une table. Enfin le script `streamlit/monitor_viz.py` permet de visualiser ces mesures de latence à l'aide d'un tableau de bord streamlit (figure 4).

Lien utile :

- `https://www.ibm.com/fr-fr/think/topics/service-level-agreement`
- `https://sre.google/sre-book/service-level-objectives/`

Pour aller plus loin :

- Mise en place d'une architecture "collecteur" + "visualiseur" en solution industriel et gratuite grâce à `Prometheus` et `Grafana`.
- Mise en place d'un système d'alerte structuré sur une fenêtre de confirmation (ex : seuil d'une métrique dépassée sur plusieurs mesures consécutives ou p95 supérieur à un seuil sur les 5 dernières minutes.) Technologies possibles : `SQL+ smtplib` ou `Prometheus` Alertmanager + `Grafana` Alerting.



FIGURE 4 – Screenshot du tableau de bord de monitoring de la base de données